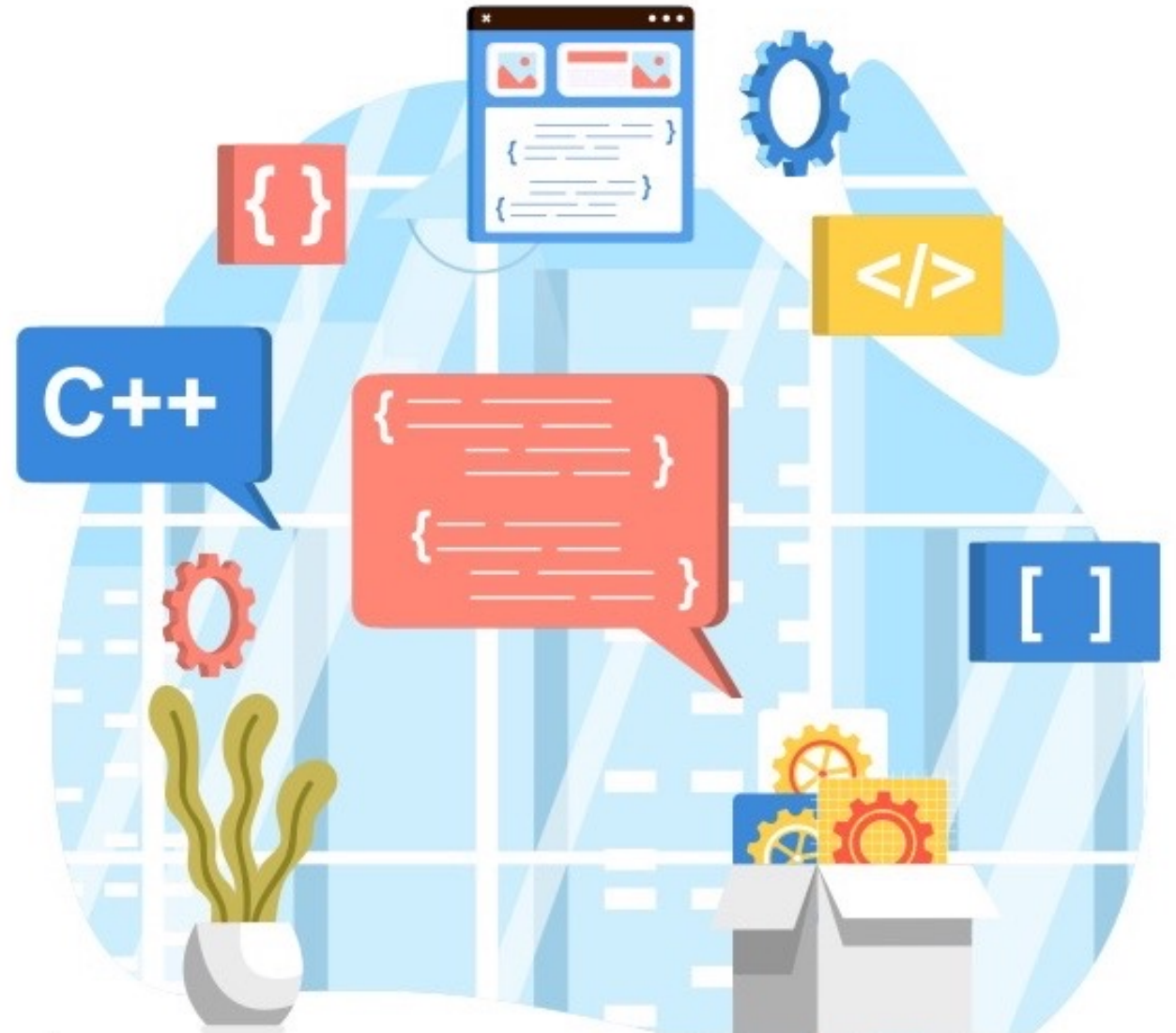


DANH SÁCH LIÊN KẾT

VŨ NGỌC THANH SANG
KHOA CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC SÀI GÒN



I - Introduction

- Dữ liệu có thể được sắp xếp và xử lý tuần tự bằng cách sử dụng mảng.
- Hạn chế khi sử dụng mảng
 - Kích thước mảng là cố định, không thể thay đổi
 - Với mảng không được sắp xếp: việc tìm kiếm phần tử xảy ra chậm
 - Mảng đã sắp xếp: việc chèn và xóa phần tử chậm vì nó yêu cầu di chuyển toàn bộ phần dữ liệu còn lại.

II - Linked Lists

- Danh sách liên kết (linked list) là một tập hợp các nút (node) bao gồm 2 thành phần: dữ liệu (data) và địa chỉ (link) của nút kế tiếp trong danh sách

data	link
------	------



II - Linked Lists

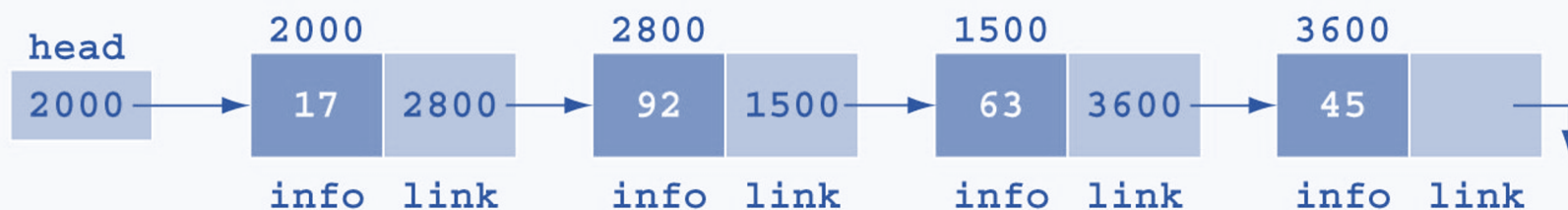
- Một nút có thể được khai báo là một lớp (class) hay một cấu trúc (struct)
- Kiểu dữ liệu của một nút phụ thuộc vào ứng dụng cụ thể.
- Thành phần liên kết của mỗi nút là một con trỏ (pointer)

```
struct nodeType
{
    int info;
    nodeType *link;
};
```

- Khai báo biến

```
nodeType *head;
```

II - Linked Lists

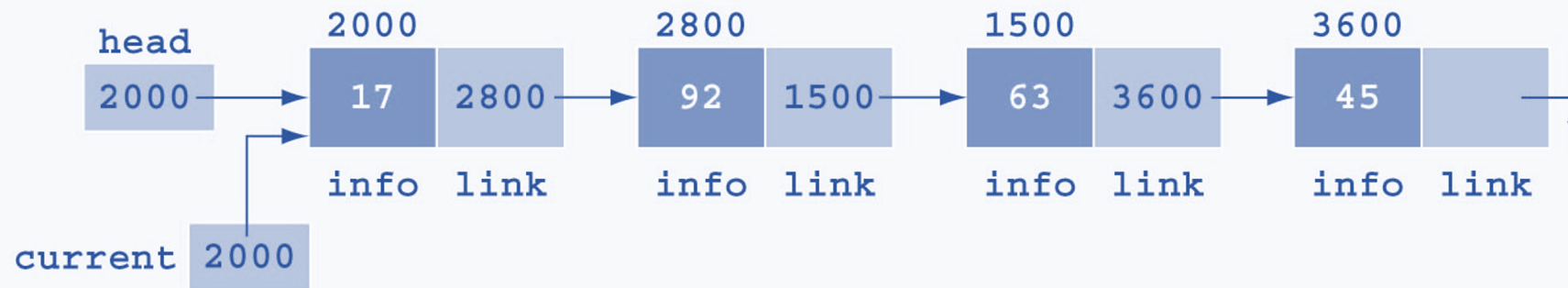


	Giá trị	Giải thích
head	2000	
head->info	17	Tại vì head có giá trị là 2000 và info tại vị trí 2000 có giá trị là 17.
head->link	2800	
head->link->info	92	Tại vì head -> link có giá trị là 2800 và info tại vị trí 2800 có giá trị là 92.

II - Linked Lists

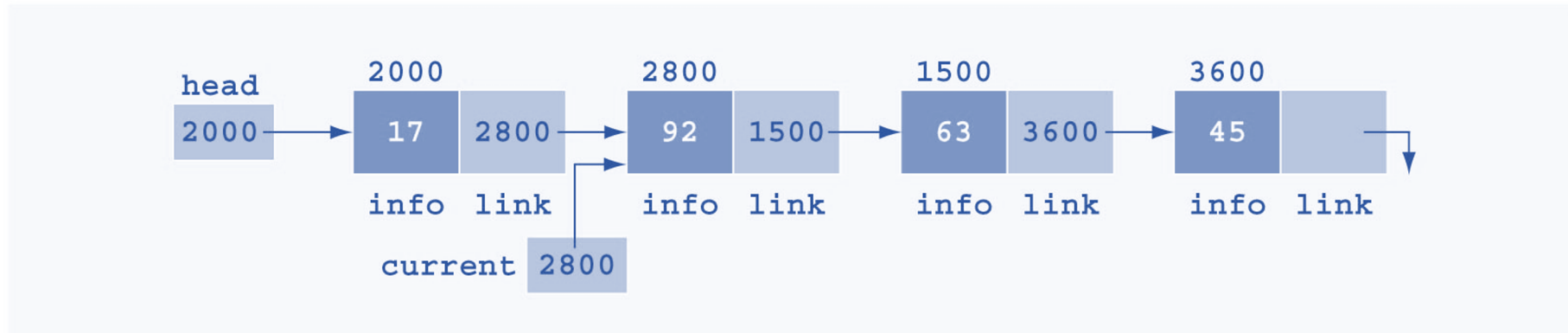
- Giả sử con trỏ **current** có cùng kiểu dữ liệu với con trỏ **head**, câu lệnh sau đây sao chép dữ liệu của con trỏ **head**:

```
current = head;
```



II - Linked Lists

```
current = current->link;
```



	Giá trị
current	2800
current->info	92
current->link	1500
current->link->info	63

II - Linked Lists

- Các thao tác cơ bản của danh sách liên kết là:
 - Tìm kiếm phần tử trong danh sách
 - Chèn phần tử vào danh sách
 - Xóa phần tử khỏi danh sách

III - Linked list Operations - Traversing a linked list

- Duyệt danh sách (traversal): là hoạt động duyệt qua từng phần tử của danh sách từ con trỏ của nút đầu tiên đến hết danh sách.
- Giả sử con trỏ **current** có cùng kiểu dữ liệu với con trỏ **head**

```
current = head;
while (current != nullptr)
{
    //Process current
    current = current->link;
}
```

III - Linked list Operations - Traversing a linked list

- Giả sử con trỏ **head** chỉ tới một danh sách liên kết, đoạn code sau in ra màn hình dữ liệu của mỗi nút.

```
current = head;
while (current != nullptr)
{
    cout << current->info << " ";
    current = current->link;
}
```

III - Linked list Operations - Item Insertion

- Giả sử con trỏ **p** trỏ tới nút có **info** là 65, để chèn một nút với **info** là 50 ở sau **p**, ta thực hiện

```
newNode = new nodeType; //create newNode
```

```
newNode->info = 50;      //store 50 in the new node
```

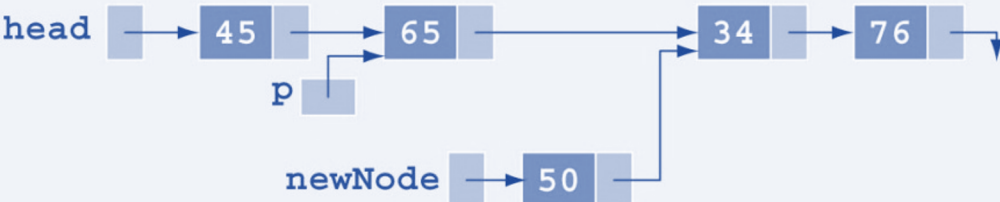
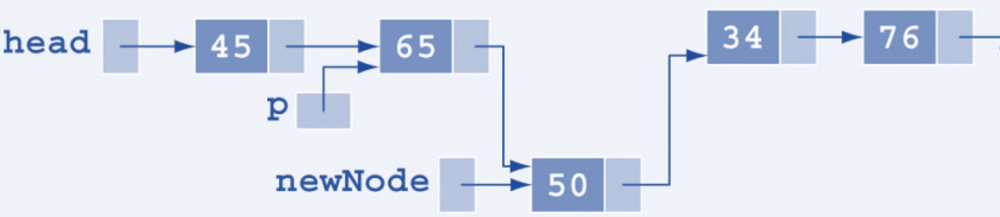
```
newNode->link = p->link;
```

```
p->link = newNode;
```

Khai báo các biến quan trọng.

```
nodeType *head, *p, *q, *newNode;
```

III - Linked list Operations - Item Insertion

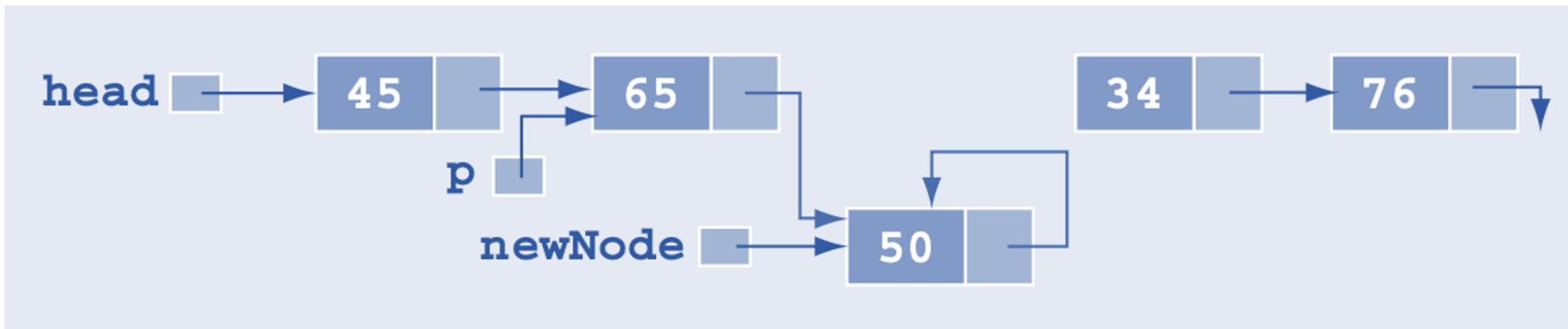
Statement	Effect
<code>newNode = new nodeType;</code>	 The diagram shows a linked list with four nodes: 45, 65, 34, and 76. A pointer 'p' points to the node containing 65. A new node 'newNode' is created with an empty data field and a null link.
<code>newNode->info = 50;</code>	 The diagram shows the linked list with nodes 45, 65, 34, and 76. The pointer 'p' still points to the node containing 65. The new node 'newNode' now contains the value 50 and has a null link.
<code>newNode->link = p->link;</code>	 The diagram shows the linked list with nodes 45, 65, 34, and 76. The pointer 'p' still points to the node containing 65. The new node 'newNode' now contains the value 50 and its link points to the node containing 34.
<code>p->link = newNode;</code>	 The diagram shows the final linked list after insertion. The new node (50) is inserted after the node containing 65. The linked list is now 45, 65, 50, 34, 76.

III - Linked list Operations - Item Insertion

- Lưu ý rằng **trình tự các câu lệnh** để chèn nút là **rất quan trọng**.
- Giả sử rằng chúng ta đảo ngược trình tự của các câu lệnh và thực hiện các câu lệnh theo thứ tự sau:

```
p->link = newNode;
```

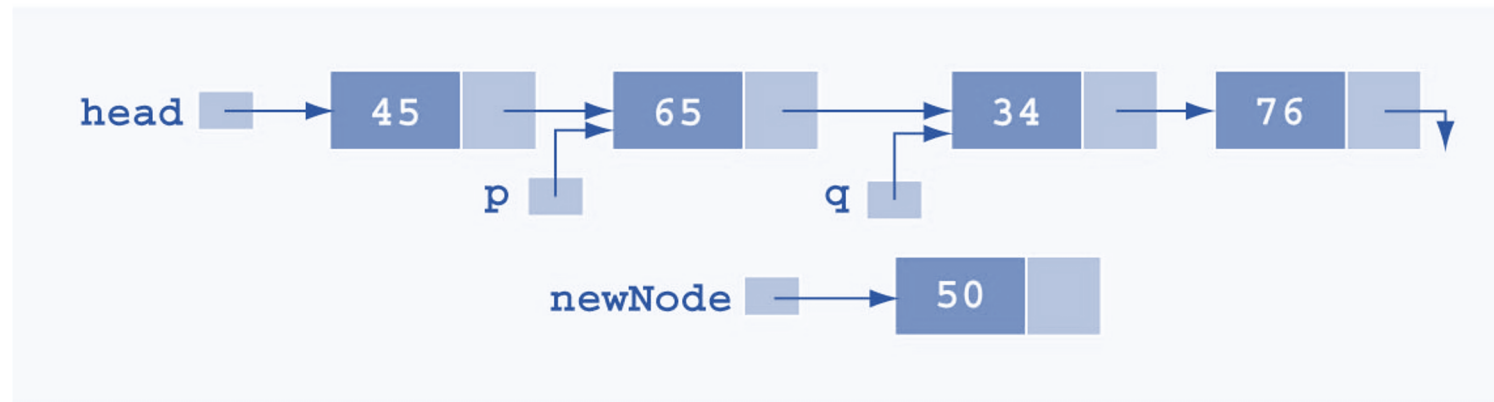
```
newNode->link = p->link;
```



- newNode trở tới chính nó và phần còn lại của danh sách bị mất.

III - Linked list Operations - Item Insertion

- Để ngăn ngừa dữ liệu bị mất khi chèn nút, ta có thể sử dụng hai con trỏ **p** và **q**.



- Để chèn newNode vào giữa p và q ta có thể thực hiện

```
newNode->link = q;  
p->link = newNode;
```

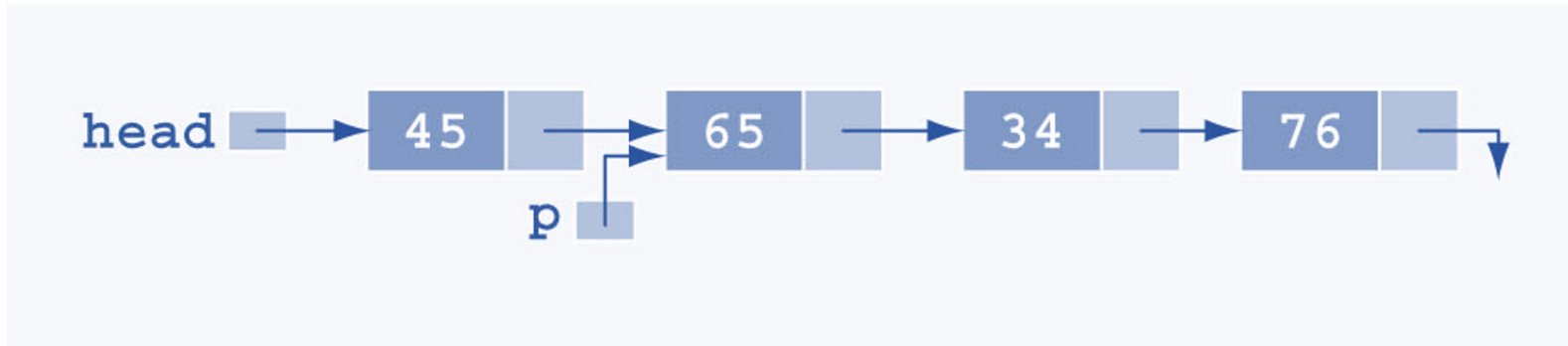
```
p->link = newNode;  
newNode->link = q;
```

III - Linked list Operations - Item Insertion

Statement	Effect
<code>p->link = newNode;</code>	The diagram illustrates the state of a linked list after the operation <code>p->link = newNode;</code>. It features two existing lists: one starting at <code>head</code> with nodes 45 and 65, and another starting at <code>q</code> with nodes 34 and 76. A new node, <code>newNode</code>, containing the value 50 is being inserted. The pointer <code>p</code> points to the node with value 45. The arrow from the <code>link</code> field of node 45 is updated to point to <code>newNode</code>. The original link of node 45, which pointed to node 65, is now broken. Node 65's <code>link</code> field still points to node 76. Node 50's <code>link</code> field is shown as an empty box, indicating it is null.
<code>newNode->link = q;</code>	The diagram illustrates the state of the linked list after the operation <code>newNode->link = q;</code>. In this step, the <code>link</code> field of <code>newNode</code> (the node with value 50) is updated to point to the head of the second list, which is <code>q</code> (the node with value 34). The rest of the structure remains the same as in the previous diagram: <code>head</code> points to 45, which points to 65, and <code>q</code> points to 34, which points to 76.

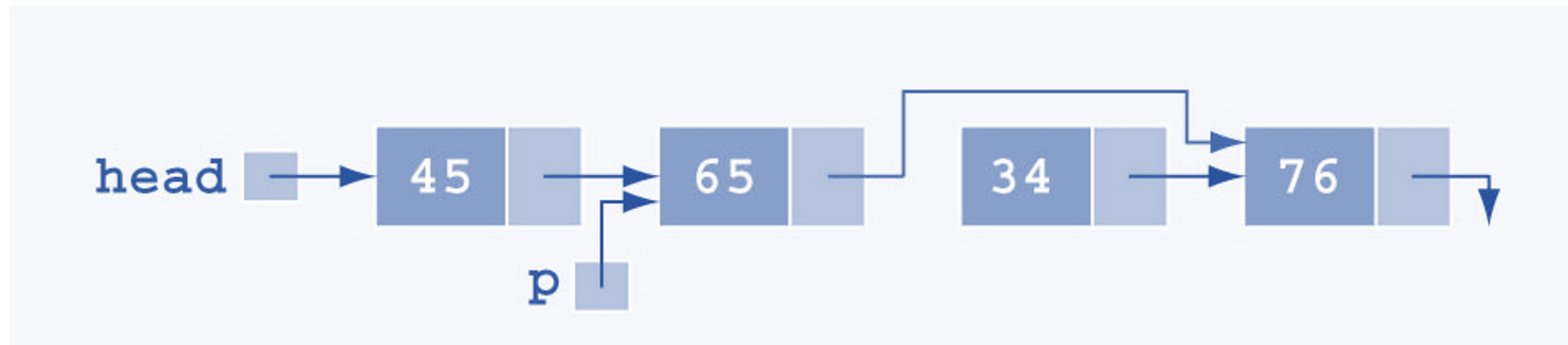
III - Linked list Operations - Item Deletion

- Xóa nút có **info** bằng 34 trong danh sách liên kết sau.



- Câu lệnh sau thực hiện yêu cầu

```
p->link = p->link->link;
```



III - Linked list Operations - Item Deletion

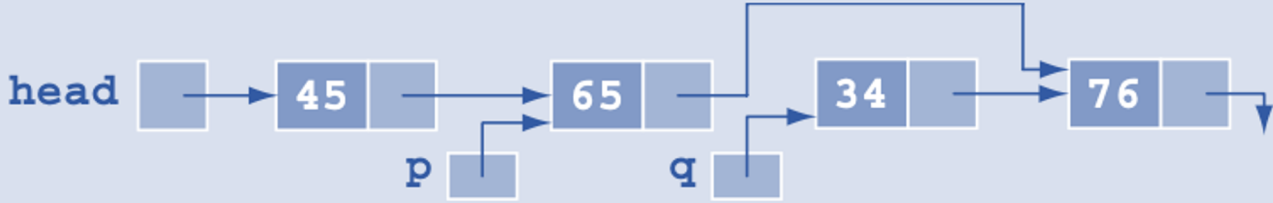
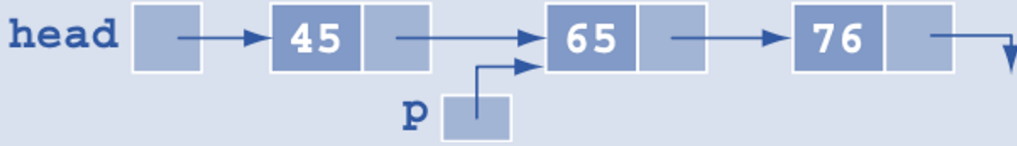
- Xóa nút có **info** bằng 34 đã bị xóa khỏi danh sách, tuy nhiên bộ nhớ vẫn bị sử dụng, the có thể nói nút đang bị treo (dangling).
- Giải pháp là sử dụng thêm một con trỏ đến nút cần xóa để phân bổ lại bộ nhớ của nút.

```
q = p->link;
```

```
p->link = q->link;
```

```
delete q;
```

III - Linked list Operations - Item Deletion

Statement	Effect
<code>q = p->link;</code>	 The diagram shows a linked list with four nodes containing values 45, 65, 34, and 76. The 'head' pointer points to the first node (45). A pointer 'p' points to the first node (45), and a pointer 'q' points to the second node (65). The nodes are connected sequentially, with the last node (76) pointing to a null terminator (represented by a downward arrow).
<code>p->link = q->link;</code>	 The diagram shows the linked list after the operation. The 'head' pointer still points to the first node (45). The pointer 'p' points to the first node (45). The pointer 'q' points to the third node (34). The link of the first node (45) now bypasses the second node (65) and points directly to the third node (34). The second node (65) is now isolated from the main chain. The third node (34) points to the fourth node (76), which points to a null terminator.
<code>delete q;</code>	 The diagram shows the final linked list after deleting the node pointed to by 'q'. The 'head' pointer points to the first node (45). The pointer 'p' points to the first node (45). The linked list now consists of three nodes: 45, 65, and 76. The link of the first node (45) points to the second node (65), which points to the third node (76). The third node (76) points to a null terminator. The node 34 has been removed from the list.

IV - Building a Linked List

- Nếu dữ liệu không được sắp xếp, danh sách liên kết cũng không được sắp xếp.
- Có thể xây dựng danh sách liên kết theo chiều xuôi (forward) hoặc chiều ngược (backward).
 - ✓ Chiều xuôi: Nút mới luôn được chèn vào cuối danh sách.
 - ✓ Chiều ngược: Nút mới luôn được chèn vào phía trước danh sách.

IV - Building a Linked List - Forward

- Cần ba con trỏ để xây dựng một danh sách
 1. Một con trỏ trỏ đến nút đầu tiên trong danh sách, không thể di chuyển
 2. Một con trỏ trỏ đến nút cuối cùng trong danh sách
 3. Một con trỏ để tạo ra nút mới
- ⇒ Xem file 6-buildListForward.txt

IV - Building a Linked List - Backward

1. Khởi tạo con trỏ first với giá trị nullptr.
2. Với mỗi phần tử trong danh sách:
 - a. Tạo một nút mới newNode
 - b. Lưu trữ giá trị trong newNode
 - c. Chèn newNode vào vị trí đầu tiên
 - d. Cập nhật giá trị của con trỏ first.

⇒ Xem file 6-buildListBackward.txt

EXERCISES

Bài 1: Viết hàm in ra giá trị của các nút trong danh sách liên kết.

Bài 2: Viết hàm in ra độ dài của danh sách liên kết.

Bài 3: Viết hàm in ra giá trị của nút đầu tiên trong danh sách liên kết.

Bài 4: Viết hàm in ra giá trị của nút cuối cùng trong danh sách liên kết.

V - Basic Operations on Linked List

1. Khởi tạo DSLK
2. Xác định xem DSLK có rỗng hay không
3. In DSLK
4. Tìm độ dài của DSLK
5. Xóa DSLK
6. Trả về/in ra giá trị của nút đầu tiên trong DSLK
7. Trả về/in ra giá trị của nút cuối cùng trong DSLK
8. Tìm kiếm phần tử trong DSLK
9. Chèn phần tử vào DSLK
10. Xóa phần tử khỏi DSLK
11. Sao chép DSLK

V - Basic Operations on Linked List

1. Khởi tạo DSLK

```
// tạo node
struct Node {
    int info; // dữ liệu tại node
    struct Node* next; // trỏ tới địa chỉ của node tiếp theo
};

int main() {
    // ...
    // khởi tạo danh sách liên kết
    struct Node* head = nullptr;
    // ...
}
```

V - Basic Operations on Linked List

2. Xác định DSLK xem có rỗng hay không

```
// xác định xem DSLK có phải rỗng hay không  
bool isEmptyList(struct Node* node){  
    return (node == nullptr);  
}
```

V - Basic Operations on Linked List

3. In DSLK

```
// in DSLK
void printList(struct Node* node) {
    // duyệt qua từng phần tử của DSLK
    while (node != nullptr) {
        cout << node->info << " "; // in ra giá trị của nút
        node = node->next; // tiếp tục trở tới nút tiếp theo
    }
    cout << endl;
}
```

V - Basic Operations on Linked List

4. Tìm độ dài của DSLK

```
// hàm xác định độ dài của DSLK
int getLength(struct Node* node) {
    int count = 0;
    // duyệt qua từng phần tử của DSLK
    while (node != nullptr) {
        count++; // đếm số phần tử
        node = node->next; // tiếp tục trở tới phần tử tiếp theo
    }
    return count;
}
```

V - Basic Operations on Linked List

5. Xóa DSLK

```
// xóa DSLK
void deleteList(struct Node* node) {
    // tạo biến tạm để lưu giá trị của phần tử cần xóa
    struct Node* temp;

    // thực hiện nếu vẫn còn nút trong DSLK
    while (node != nullptr){
        temp = node;           // lấy địa chỉ của nút
        node = node->next;     // trở tới nút tiếp theo
        delete temp;          // xóa giá trị tại địa chỉ temp
    }
}
```

V - Basic Operations on Linked List

6. Trả về giá trị của nút đầu tiên trong DSLK

```
// trả về giá trị của nút đầu tiên trong DSLK  
void getFirstNode(struct Node* node) {  
    cout << node->info << endl;  
}
```

V - Basic Operations on Linked List

7. Trả về giá trị của nút cuối cùng trong DSLK

```
// hàm trả về giá trị của nút cuối cùng trong DSLK
void getLastNode(struct Node* node) {
    // duyệt tới khi hết mảng
    while (node->next != nullptr) {
        node = node->next;
    }
    // in ra giá trị của nút cuối cùng
    cout << node->info << endl;
}
```

V - Basic Operations on Linked List

8. Tìm kiếm phần tử trong DSLK chưa sắp xếp

```
// hàm tìm kiếm phần tử trong DSLK
bool searchItem(struct Node* node, int& searchItem)
{
    struct Node *current; // tạo con trỏ để duyệt DSLK
    bool found = false; // tạo cờ để xác định quá trình tìm kiếm

    current = node; // con trỏ chỉ tới phần tử đầu tiên

    while (current != nullptr && !found) // tìm kiếm trong danh sách
        if (current->info == searchItem) // nếu tìm thấy phần tử
            found = true;
        else // nếu không tìm thấy
            current = current->next; // trỏ tới nút tiếp theo

    return found;
}
```

V - Basic Operations on Linked List

9.1. Chèn phần tử vào vị trí đầu tiên trong DSLK

```
// hàm chèn nút tại vị trí đầu tiên
void insertAtBeginning(struct Node** head_ref, int new_data) {

    struct Node *newNode; // tạo con trỏ chỉ tới nút mới

    newNode = new Node;    // tạo nút mới

    // chèn dữ liệu
    newNode->info = new_data;
    newNode->next = (*head_ref);

    // trỏ tới địa chỉ của nút mới được chèn
    (*head_ref) = newNode;
}
```

V - Basic Operations on Linked List

9.2. Chèn phần tử vào vị trí cuối cùng trong DSLK

```
// hàm chèn nút tại vị trí cuối cùng
void insertAtEnd(struct Node** head_ref, int new_data) {
    struct Node* new_node; // tạo nút mới
    new_node = new Node;
    struct Node* last = *head_ref;
    // cập nhật giá trị cho nút mới
    new_node->info = new_data;
    new_node->next = nullptr;

    if (*head_ref == nullptr) { // nếu danh sách rỗng thì trở tới nút mới
        *head_ref = new_node;
        return;
    }

    while (last->next != nullptr) // duyệt tới nút cuối cùng
        last = last->next;
    last->next = new_node; // nút cuối cùng chỉ tới nút mới
    return;
}
```

Bài tập

1. Viết chương trình chèn phần tử vào một vị trí thứ n trong sách sách liên kết.
2. Viết chương trình để xóa một phần tử khỏi danh sách liên kết

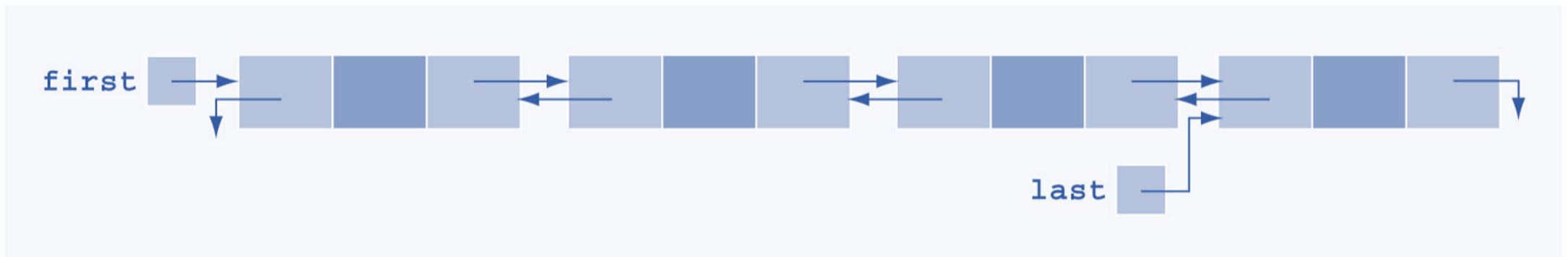
VI - Double Linked List

- Hạn chế của DSLK đơn đó là khi xóa nút ở vị trí cuối cùng, ta phải thực hiện việc duyệt hết DSLK.
- Khi kích thước DSLK lớn, thao tác này sẽ được thực hiện rất chậm.
- Giải pháp đó chính là tái cấu trúc nút để có thể vừa chứa địa chỉ của nút trước nó và nút sau nó.
- Đây gọi là DSLK kép.

VI - Double Linked List

- Danh sách liên kết kép: tại mỗi nút có hai con trỏ để chỉ đến nút tiếp theo (chiều xuôi) và nút trước đó (chiều ngược),
- Có thể duyệt mảng theo hai chiều.

```
struct Node
{
    int info;
    Node *next;
    Node *back;
};
```



VI - Double Linked List - Operations

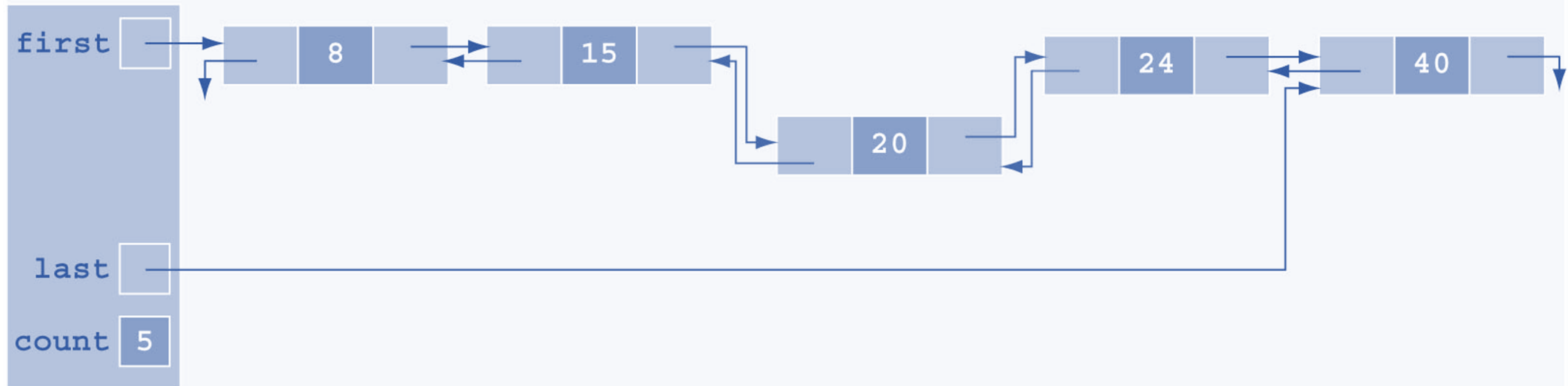
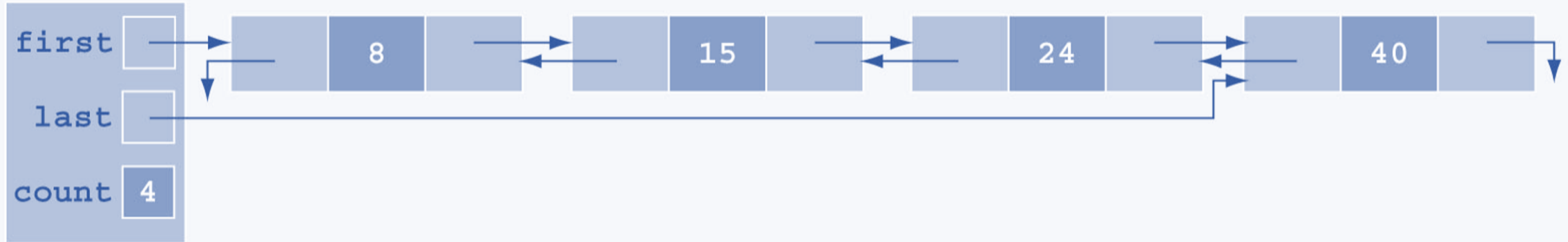
1. Khởi tạo DSLK kép
2. Xác định xem DSLK có rỗng hay không
3. In DSLK
4. Tìm độ dài của DSLK
5. Trả về/in ra giá trị của nút đầu tiên/ cuối cùng trong DSLK
6. Tìm kiếm phần tử trong DSLK
7. Chèn phần tử vào DSLK
8. Xóa phần tử khỏi DSLK
9. Sao chép DSLK

VI - Double Linked List - Node Insertion

Các trường hợp chèn nút:

- TH1: Chèn nút vào DSLK rỗng
- TH2: Chèn nút vào vị trí đầu tiên của DSLK không rỗng
- TH3: Chèn nút vào vị trí cuối cùng của DSLK không rỗng
- TH4: Chèn nút vào vị trí bất kì của DSLK không rỗng

VI - Double Linked List - Node Insertion

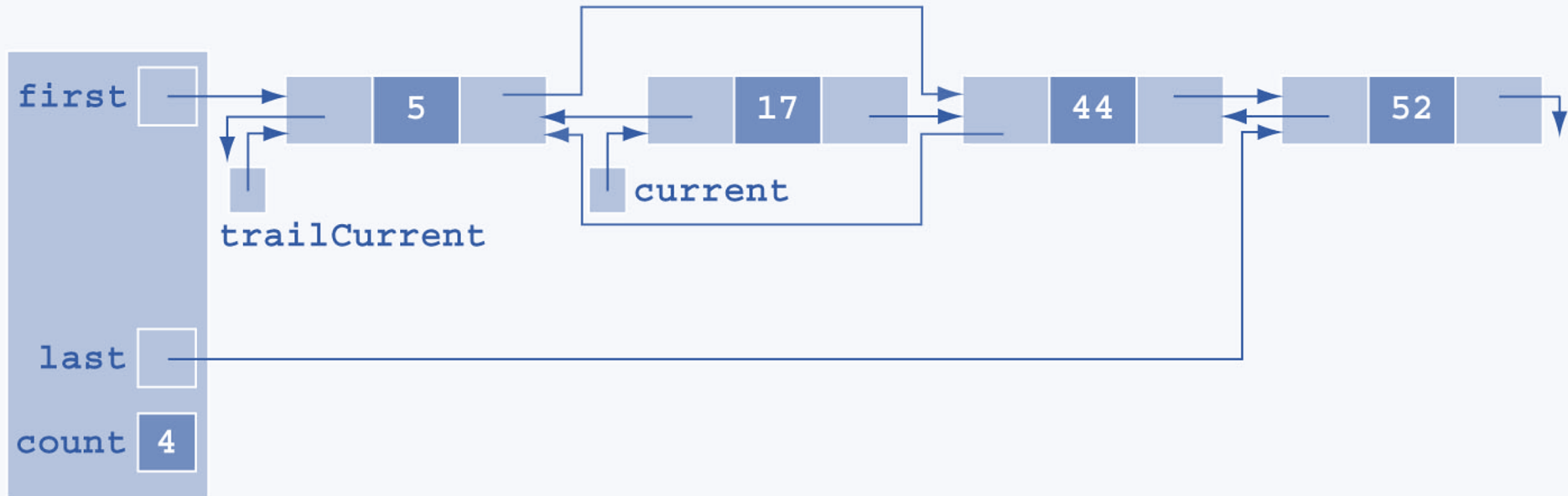
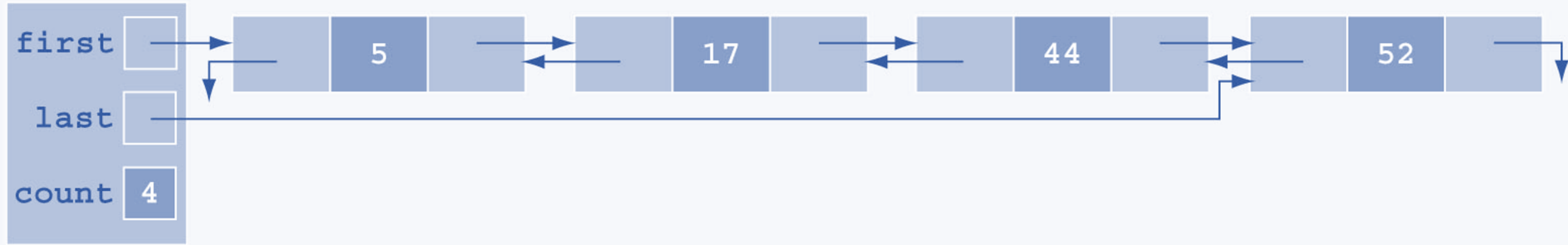


VI - Double Linked List - Node Deletion

Các trường hợp chèn nút:

- TH1: Xóa nút của DSLK rỗng
- TH2: Xóa nút tại vị trí đầu tiên của DSLK không rỗng
- TH3: Xóa nút tại vị trí cuối cùng của DSLK không rỗng
- TH4: Xóa nút tại vị trí bất kì của DSLK không rỗng
- TH5: Xóa nút không tồn tại trong DSLK

VI - Double Linked List - Node Insertion

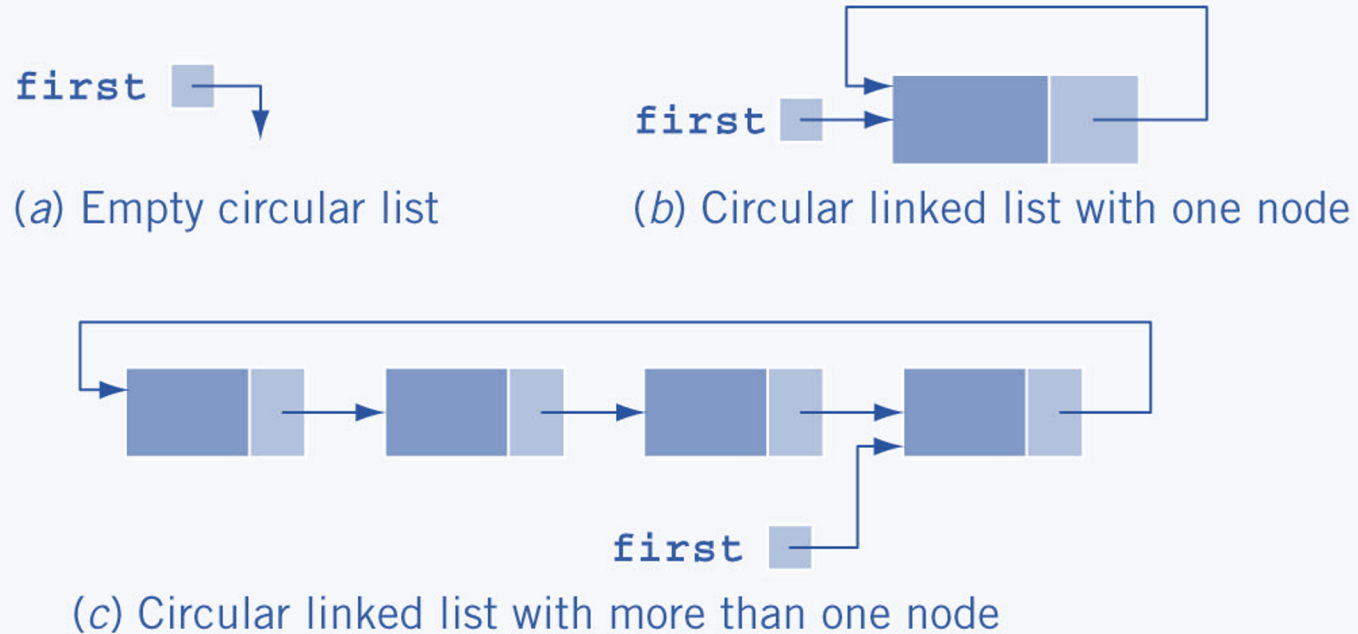


VII – Circular Linked List

1. Khởi tạo DSLK kép
2. Xác định xem DSLK có rỗng hay không
3. In DSLK
4. Tìm độ dài của DSLK
5. Trả về/in ra giá trị của nút đầu tiên/ cuối cùng trong DSLK
6. Tìm kiếm phần tử trong DSLK
7. Chèn phần tử vào DSLK
8. Xóa phần tử khỏi DSLK
9. Sao chép DSLK

VII - Circular Linked List

- Danh sách liên vòng: con trỏ tại nút cuối cùng của DSLK trở tới địa chỉ của nút đầu tiên trong DSLK,



VII - Circular Linked List - Operations

1. Khởi tạo DSLK vòng
2. Xác định xem DSLK có rỗng hay không
3. In DSLK
4. Tìm độ dài của DSLK
5. ~~Trả về/in ra giá trị của nút đầu tiên/ cuối cùng trong DSLK~~
6. Tìm kiếm phần tử trong DSLK
7. Chèn phần tử vào DSLK
8. Xóa phần tử khỏi DSLK
9. Sao chép DSLK

1. Viết các hàm thực hiện và đánh giá các thao tác cơ bản của DSLK kép.
2. Viết các hàm thực hiện và đánh giá các thao tác cơ bản của DSLK vòng.

THANK YOU FOR YOUR ATTENTIONS